

Existing monitoring capabilities

- List of metrics to monitor
 - Databricks
 - Fivetran
 - AWS
- Metrics monitoring limitations
- Already in place monitoring tools
 - Databricks built in tools
 - Billing
 - Spark UI
 - Job run details
 - Cluster logs
 - Audit Logs
 - Hive metastore
 - System resource-level metrics
 - Fivetran built in tools
 - Fivetran dashboard
 - External logging services
 - Fivetran Log Connector
 - AWS
 - S3 Buckets
 - Metrics
 - Audit logging
 - EC2
 - Networking
- Conclusion
 - Fivetran
 - Suggested action items
 - Databricks
 - Suggested action items
 - AWS
 - Suggested action items

List of metrics to monitor

Databricks

- Job run details
 - duration
 - id
 - status



- cluster config
- Clusters (both job clusters and users clusters)
 - Health check
 - Logs
 - Driver logs
 - Executor logs
 - Event logs
 - System resource-level metrics (ganglia APM)
 - CPU
 - Memory
 - Disk usage
 - Network
 - Spark metrics
 - Spark UI
- Hive metastore
- Audit logs

Fivetran

- Usage/consumption
 - MAR
- User actions
 - Historical re-syncs
 - Pausing/resuming connectors
 - Changing sync cycles
- Sync
 - Run time
 - Status
 - Volume
 - Logs

AWS

- Buckets
 - Metrics (Storage Lens)
 - Audit logging
 - S3 server access logging
 - S3 object level logging
- EC2
 - APM
 - CPU
 - Disk

- Memory
- Network
- Status check
- Networking
 - VPC flow logs
 - Firewall flow logs
- AWS audit logs
 - Configure per service

Metrics monitoring limitations

Tool	Metric	Built in tools dashboards limitations	S3 storage integrations	Cloudwatch integrations
Databricks	Job run details	60 days only	Job run logs can be stored on S3 but nothing for the details of the job such as cluster config	No integration
	Clusters logs	30 days only Slow when cluster has been terminated Not centralized No alerting based on log events	Can be stored in S3 for long term storage	Cloudwatch agent captures and monitors these logs for as long as we want We can implement alerts
	Spark metrics *		No integration	Cloudwatch agent should be able to capture these metrics as well but that needs to be verified We can implement alerts
	System resource-level metrics	30 days only Not complete APM when cluster has been terminated		Cloudwatch agent captures and monitors these metrics for as long as we want

		Not centralized		We can implement alerts
		No alerting		
	Hive metastore audit	No alerting based on log events. Audit logs are saved on S3		No integration
	Audit logs			
Fivetran	Monthly active rows	None	These type of metrics can be persisted indefinitely inside of databricks. Still no alerting	No integration
	User actions	7 days only No alerting		No integration
	Sync metrics	7 days only		Can be forwarded to Cloudwatch to set up alerting
AWS	S3 Metrics	Limited to 14 days		
	S3 server access logs	Logs are kept as a file on a S3 bucket		No integrations
	S3 object level logging	Logs are sent directly to Cloudwatch		
	EC2 APM		Logs can be pushed to S3	Cloudwatch agent can collect these APM and logs
	VPC flow logs	Logs are sent directly to Cloudwatch		
	Firewall flow logs			
	Audit logs			

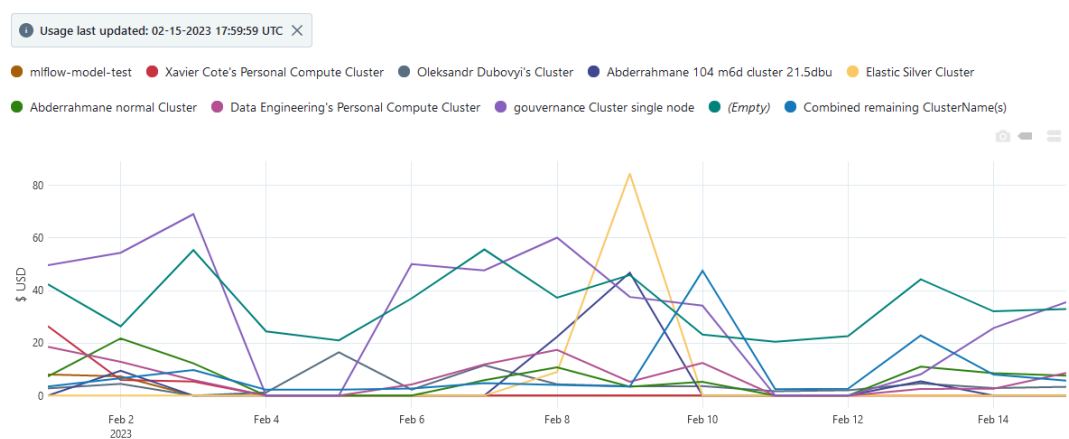
Already in place monitoring tools

Databricks built in tools

Billing

Databricks does offer internal tools to be able to drill down costs and monitor our usage of the databricks platform over time. Databricks also offers the

possibility of exporting the usage to a CSV file on a S3 bucket.



Limitations

There doesn't seem to be the capability of being able to send an alert once a certain threshold has been reached.

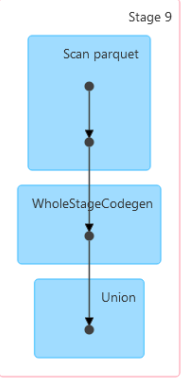
Spark UI

The Spark UI gives us plenty of information to better understand what is being ran on a server. From this UI we can see queries and their explain plan, jobs and their metrics (shuffle, skew, etc). It is the main interface we use to debug performance issues.

Open in new tab

Jobs Stages Storage Environment Executors SQL / DataFrame JDBC/ODBC Server Structured Streaming

Stage 9



▼ Completed Stages (1)

Page: 1 1 Pages. Jump to 1 . Show 100 items in a page. Go

Stage Id	Pool Name	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
9	6985535514833124986	select * from `default`.`spypoint_keys_temp` executeCollect at OptimizeLocalUnion.scala:71 + details	2023/02/15 19:06:36	5 s	1/1	18.3 KiB			

Limitations

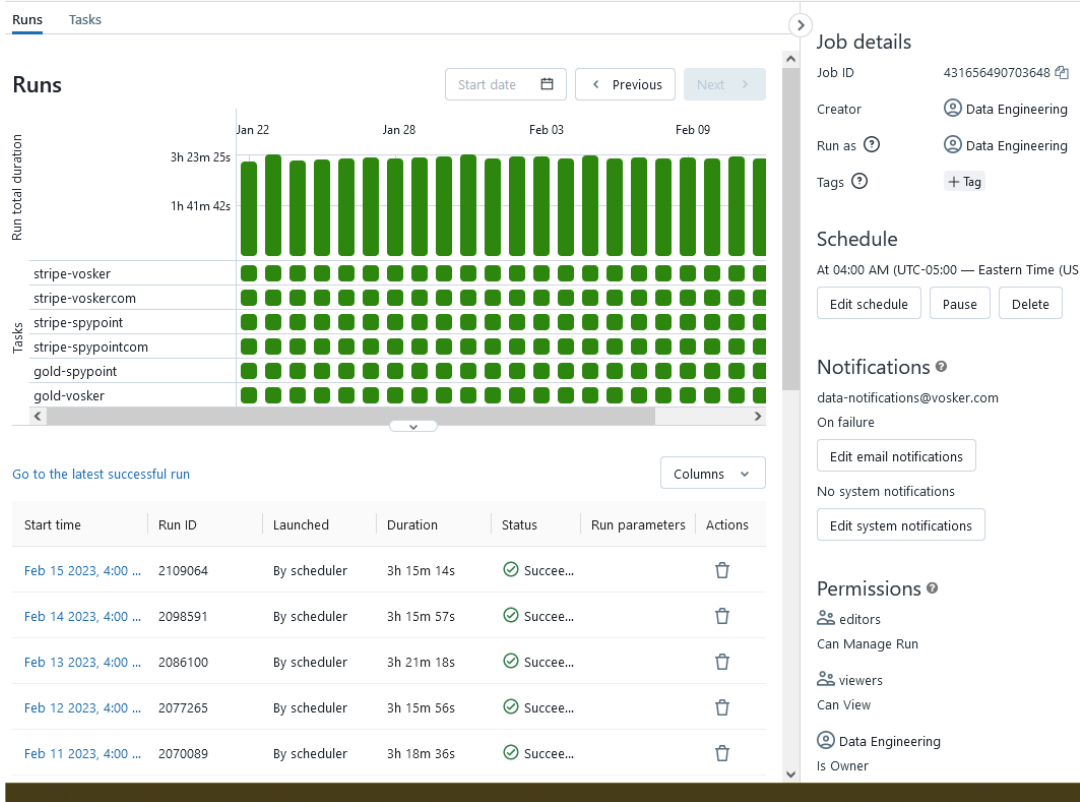
The Spark-UI works great on live servers but takes multiple minutes (seen up to 30 minutes) to retrieve the log files and these log files are only kept a certain amount of time (30 days).

The Spark-UI must be consulted on a per cluster basis.

Job run details

Databricks does offer different tools to be able to get more information about our jobs. The interface allows you to quickly see all execution of a specific job their duration and status.

[Stripe] pipeline job



Further information about specifics runs can also be found when we select a specific job run such as Cluster used with links to its metrics ,spark-UI, configuration and job parameters.

Task run details

Job ID	1067971352528446 
Job run ID	1206268 
Task run ID	1206965 
Run as	 Oleksandr Dubovyi
Started	2023-02-09 13:21:23 EST
Ended	2023-02-09 13:26:48 EST
Duration	5m 24s
Status	 Succeeded

Notebook

[/Repos/odubovyi@vosker.com/integration-test/src/notebooks/bronze/test_init_bronze](#) 

Compute

● IT_cluster

Driver: i3.xlarge · Workers: i3.xlarge · 2-4 workers · On-Demand · 10.4 LTS (includes Apache Spark 3.2.1, Scala 2.12) · us-east-1a

[View cluster](#)

[Spark UI](#)

[Logs](#)

[Metrics](#)

Parameters

Limitations

Other than the limitations noted in the other sections this part of databricks seems to be able to correspond to our needs.

Cluster logs

Cluster logs are saved for 30 days by default and they are divided into 3 main files.

Spark driver logs — Recent log files

☒ Auto-fetch data

Log file	Log type	Size	Run as	Duration	Status	Time
stdout	Standard output	432.38 KB	Oleksandr Dubovyi	Started		Latest log
stderr	Standard error	1.36 KB		2023-02-09 13:26		Latest log
log4j-active.log	Log4j output	4.73 MB		3m 24s	Succeeded	Latest log

Job run ID: 1206266

Search ID: 1067871352508446

Notebook: /Repos/odubovyi@vosker.com/integration-test/src/notebooks/bronze/test mit bronze

These logs files can be persisted on a S3 bucket at the location that we specify. This will ensure that the logs are kept long term.

Spark SSH Logging

Destination ? Cluster Log Path ?

DBFS ⬆️⬆️ dbfs:/cluster-logs

Audit Logs

Databricks provides access to audit logs of activities performed by Databricks users, allowing your enterprise to monitor detailed Databricks usage patterns.

There are two types of logs:

- Workspace-level audit logs with workspace-level events.
- Account-level audit logs with account-level events.

Here is a list of all the events : [Audit log reference](#) | [Databricks on AWS](#)

Once enabled we can configure it to add these actions if we turn on verbose mode

- Each time a user runs a command from a notebook
- Each time a user submits a query from the SQL interface
- Each time a SQL interface query is completed or canceled.



These logs are saved on a S3 bucket. Here is an example of what the audit logs even look like:

```
1  {
2    "version":"2.0",
3    "auditLevel":"ACCOUNT_LEVEL",
4    "timestamp":1629775584891,
5    "orgId":"3049056262456431186970",
6    "shardName":"test-shard",
7    "accountId":"77636e6d-ac57-484f-9302-f7922285b9a5",
8    "sourceIPAddress":"10.2.91.100",
9    "userAgent":"curl/7.64.1",
10   "sessionId":"ephemeral-f836a03a-d360-4792-b081-baba525324312",
11   "userIdentity":{
12     "email":"crampton.rods@email.com",
13     "subjectName":null
14   },
15   "serviceName":"unityCatalog",
16   "actionName":"createMetastoreAssignment",
17   "requestId":"ServiceMain-da7fa5878f40002",
18   "requestParams":{
19     "workspace_id":"30490590956351435170",
20     "metastore_id":"abc123456-8398-4c25-91bb-b000b08739c7",
21     "default_catalog_name":"main"
22   },
23   "response":{
24     "statusCode":200,
25     "errorMessage":null,
26     "result":null
27   },
28   "MAX_LOG_MESSAGE_LENGTH":16384
29 }
```

Databricks delivers a separate JSON file for each workspace in your account and a separate file for account-level events.

These audit logs can then be queried directly from databricks.

Limitations

The log delivery functionality is only available from a post call.

There is a limit on the number of log delivery configurations available per account (each limit applies separately to each log type, including billable usage and audit logs). You can create a maximum of two enabled account-level delivery configurations (configurations without a workspace filter) per type. Additionally, you can create two enabled workspace level delivery configurations per workspace for each log type, meaning the same

workspace ID can occur in the workspace filter for no more than two delivery configurations per log type. You cannot delete a log delivery configuration, but you can disable it. You can re-enable a disabled configuration, but the request fails if it violates the limits previously described.

Hive metastore

Hive metastore monitoring or auditing only seems to be available via Unity Catalog.

This article contains audit log information for Unity Catalog events. Unity Catalog captures an audit log of actions performed against the metastore. This enables admins to access fine-grained details about who accessed a given dataset and what actions they performed.

To be implemented we need Unity Catalog and to already have audit logging active.

Limitations

Restricted to Unity Catalog

System resource-level metrics

Limitations

- Ganglia doesn't support alerts.
- Ganglia doesn't support creating metrics derived from logs (e.g., ERROR log growth rate).
- You can't use custom dashboards to track SLO (Service Level Objectives) and SLI (Service Level Indicators) related to data-correctness, data-freshness, or end-to-end latency, and then visualize them with ganglia.
- Retrieving the log files for a closed cluster takes a long time
- After more than 30 days the data is not available
- Ganglia metrics after the cluster is close is only screenshots of the dashboard

job-1067971352528446-run-1206268-IT_cluster

[Configuration](#) [Libraries](#) [Event log](#) [Spark UI](#) [Driver logs](#) [Metrics](#) [Apps](#) [Spark cluster UI - Master](#) ▼

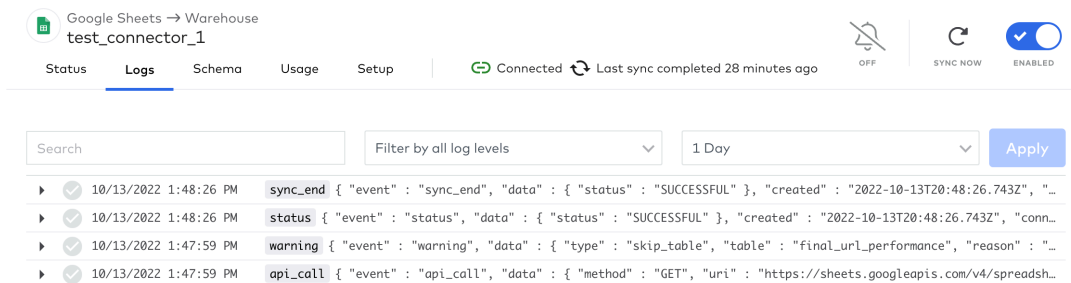
Loading

Loading old UI for cluster "0209-182125-ces7636s"... (collecting log files) This may take a few minutes.

Fivetran built in tools

Fivetran dashboard

Logs about our syncs per destination are easily found via the Fivetran dashboard. From this page we can filter by error type and more to be able to better understand what is happening.



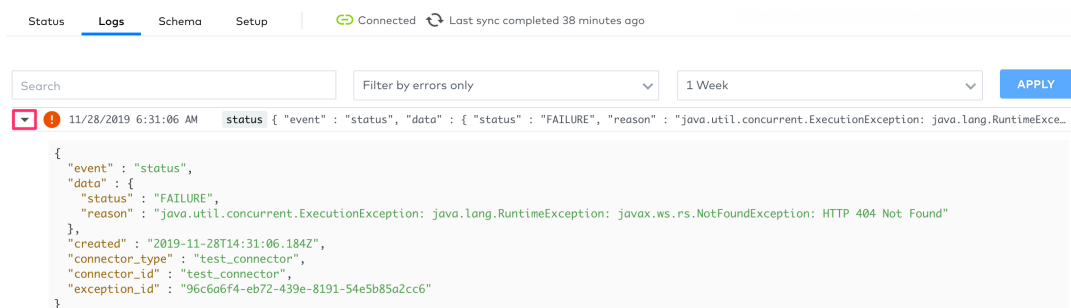
Google Sheets → Warehouse
test_connector_1

Status **Logs** Schema Usage Setup Connected Last sync completed 28 minutes ago OFF SYNC NOW ENABLED

Search Filter by all log levels 1 Day Apply

- 10/13/2022 1:48:26 PM **sync_end** { "event" : "sync_end", "data" : { "status" : "SUCCESSFUL" }, "created" : "2022-10-13T20:48:26.743Z", "...
- 10/13/2022 1:48:26 PM **status** { "event" : "status", "data" : { "status" : "SUCCESSFUL" }, "created" : "2022-10-13T20:48:26.743Z", "conn...
- 10/13/2022 1:47:59 PM **warning** { "event" : "warning", "data" : { "type" : "skip_table", "table" : "final_url_performance", "reason" : "...
- 10/13/2022 1:47:59 PM **api_call** { "event" : "api_call", "data" : { "method" : "GET", "uri" : "https://sheets.googleapis.com/v4/spreadsh...

Example of the logs available from within Fivetran

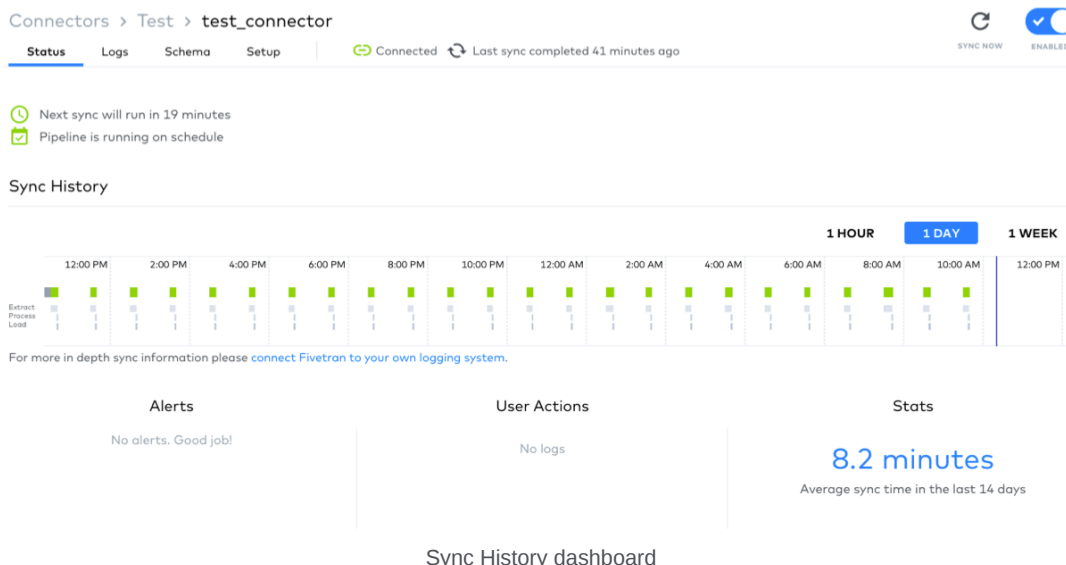


Status **Logs** Schema Setup Connected Last sync completed 38 minutes ago

Search Filter by errors only 1 Week APPLY

11/28/2019 6:31:06 AM **status** { "event" : "status", "data" : { "status" : "FAILURE", "reason" : "java.util.concurrent.ExecutionException: java.lang.RuntimeException: javax.ws.rs.NotFoundException: HTTP 404 Not Found" }, "created" : "2019-11-28T14:31:06.184Z", "connector_type" : "test_connector", "connector_id" : "test_connector", "exception_id" : "96c6a6f4-eb72-439e-8191-54e5b85a2cc6" }

Details of a error



However only 1 week of data is available from this dashboard. Fivetran recommends to keep its logs externally via its log connector if we want to keep the information long-term.

External log services can retain connector log data longer than Fivetran. **We retain connector logs for one week**, so we recommend that you connect to an external log service if you want to access logs older than seven days. Logging services usually allow users to keep log data for several months at a minimum, though the specifics vary by provider. IMPORTANT: You must have an Enterprise or Business Critical plan to connect an external logging service.

In Fivetran, a notification is an automatically-generated email. It quickly informs you about important activity in your Fivetran account, like connector failures and any other types of errors, so that you can resolve any potential issues as soon as possible. These alerts are specific to each user!

External logging services

Using Fivetran's AWS Cloudwatch Log service, you can:

- Connect and stream Fivetran log events to Cloudwatch
- Manage the life cycle of the log events using Cloudwatch's configuration
- Search and filter the logs
- Setup monitoring or alerting mechanisms based on log events



Fivetran supports the following logging services:

- [AWS CloudWatch](#)
- [Azure Log Analytics](#)
- [Datadog](#)
- [Google Cloud Logging](#)
- [Splunk](#)

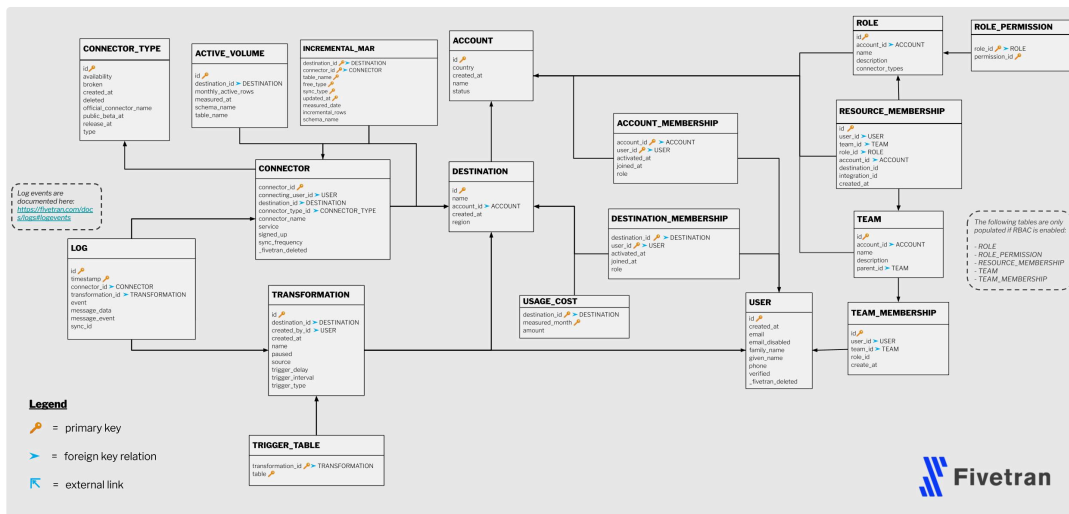
Setup consist in created a IAM role policy that gives access to our Cloudwatch to Fivetran external account in a similar way we had to configure for databricks.

Fivetran Log Connector

Fivetran generates structured log events from connectors, dashboard user actions, and Fivetran API calls. The Fivetran Log Connector delivers your logs and account metadata to a schema in your destination. This metadata includes granular Fivetran consumption information.

Fivetran gives us the option of storing the logs and the metrics directly into our destination without additional cost on their end. The ERD diagram of what data available is found below but this would allow us to be able to query and analyze our Fivetran setup from within databricks itself.

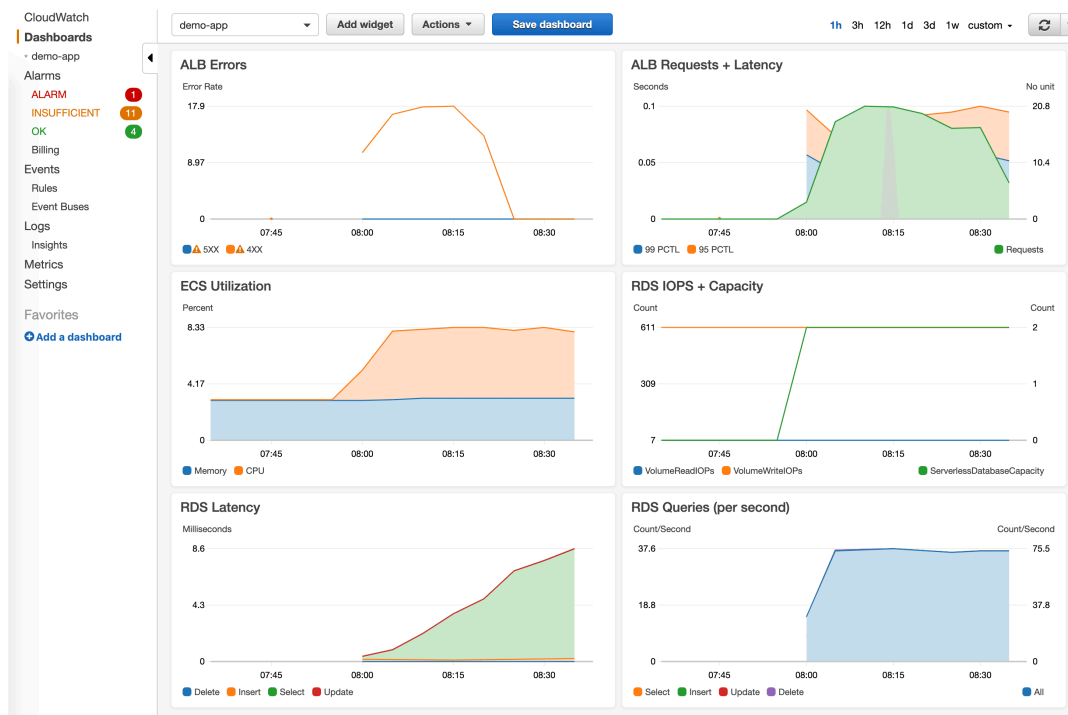
We could then build dashboards from within databricks to be able to have different metrics.



ERD of all the data generated by the fivetran log connector

AWS

Cloudwatch offers us a way to centralize monitoring into one tool. It allows to collect logs and metrics from different services inside of the AWS cloud and allows external services such as Fivetran and Databricks to send their logs and metrics as well. This centralized information can then be viewed from the interface and via dashboards.

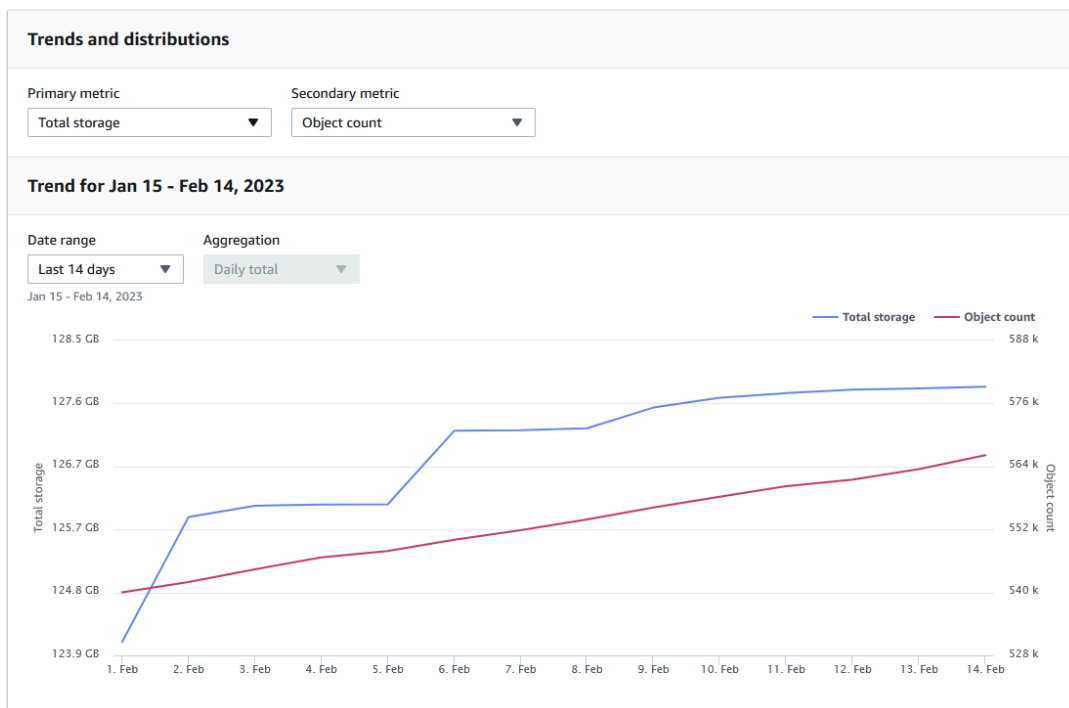


Example of a Cloudwatch dashboard

S3 Buckets

Metrics

We currently can access to Storage lens dashboard that shows us a few metrics about our buckets.



However this dashboard is limited to two weeks of data so if we would like to see the growth over a longer period of time then this built in tool does not meet our needs.

Thankfully we can forward these metrics to Cloudwatch to persist them longer.

Audit logging

AWS offers two different mode of logging for S3 auditing. Both of these can be combined and serve different purposes : [Logging options for Amazon S3](#)

[3 - Amazon Simple Storage Service](#)

AWS Cloudtrail allows its logs to further be redirected to Cloudwatch while the S3 server logs are not.

The Cloudtrail option does not solves all of our monitoring needs as actions like Lifecycle transitions, expiration and restores are only logged to the S3 server logs.

So we would need potentially need to enable both services depending on how and what we would like to log.

S3 Cloudtrail log example :

```
1  {
2    "Records": [
3      {
4        "eventVersion": "1.03",
5        "userIdentity": {
6          "type": "IAMUser",
7          "principalId": "111122223333",
8          "arn": "arn:aws:iam::111122223333:user/myUserName",
9          "accountId": "111122223333",
10         "accessKeyId": "AKIAIOSFODNN7EXAMPLE",
11         "userName": "myUserName"
12       },
13       "eventTime": "2019-02-01T03:18:19Z",
14       "eventSource": "s3.amazonaws.com",
15       "eventName": "ListBuckets",
16       "awsRegion": "us-west-2",
17       "sourceIPAddress": "127.0.0.1",
18       "userAgent": "[]",
19       "requestParameters": {
20         "host": [
21           "s3.us-west-2.amazonaws.com"
22         ]
23       },
24       "responseElements": null,
25       "additionalEventData": {
26         "SignatureVersion": "SigV2",
27         "AuthenticationMethod": "QueryString",
28         "aclRequired": "Yes"
29     },
30     "requestID": "47B8E8D397DCE7A6",
31     "eventID": "cdc4b7ed-e171-4cef-975a-ad829d4123e8",
32     "eventType": "AwsApiCall",
33
34     "recipientAccountId": "444455556666"
35   },
36   {
37     "eventVersion": "1.03",
38     "userIdentity": {
39       "type": "IAMUser",
40       "principalId": "111122223333",
41       "arn": "arn:aws:iam::111122223333:user/myUserName",
42       "accountId": "111122223333",
43       "accessKeyId": "AKIAIOSFODNN7EXAMPLE",
44       "userName": "myUserName"
45     },
46     "eventTime": "2019-02-01T03:22:33Z",
47     "eventSource": "s3.amazonaws.com",
48     "eventName": "PutBucketAcl",
49     "awsRegion": "us-west-2",
50     "sourceIPAddress": "",
51     "userAgent": "[]",
```

```

52     "requestParameters": {
53         "bucketName": "",
54         "AccessControlPolicy": {
55             "AccessControlList": {
56                 "Grant": {
57                     "Grantee": {
58                         "xsi:type": "CanonicalUser",
59                         "xmlns:xsi": "http://www.w3.org/2001/XMLSchema-",
60                         "ID": "d25639fbe9c19cd30a4c0f43fbf00e2d3f96400a9aa8dabf",
61                     },
62                     "Permission": "FULL_CONTROL"
63                 }
64             },
65             "xmlns": "http://s3.amazonaws.com/doc/2006-03-01/",
66             "Owner": {
67                 "ID": "d25639fbe9c19cd30a4c0f43fbf00e2d3f96400a9aa8dabf",
68             }
69         }
70         "host": [
71             "s3.us-west-2.amazonaws.com"
72         ],
73         "acl": [
74             ""
75         ]
76     },
77     "responseElements": null,
78     "additionalEventData": {
79         "SignatureVersion": "SigV4",
80         "CipherSuite": "ECDHE-RSA-AES128-SHA",
81         "AuthenticationMethod": "AuthHeader"
82     },
83     "requestID": "BD8798EACDD16751",
84     "eventID": "607b9532-1423-41c7-b048-ec2641693c47",
85     "eventType": "AwsApiCall",
86     "recipientAccountId": "111122223333"
87 },
88 {
89     "eventVersion": "1.03",
90     "userIdentity": {
91         "type": "IAMUser",
92         "principalId": "111122223333",
93         "arn": "arn:aws:iam::111122223333:user/myUserName",
94         "accountId": "111122223333",
95         "accessKeyId": "AKIAIOSFODNN7EXAMPLE",
96         "userName": "myUserName"
97     },
98     "eventTime": "2019-02-01T03:26:37Z",
99     "eventSource": "s3.amazonaws.com",
100    "eventName": "GetBucketVersioning",
101    "awsRegion": "us-west-2",
102    "sourceIPAddress": "",
103    "userAgent": "[]",
104    "requestParameters": {
105        "host": [
106            "s3.us-west-2.amazonaws.com"
107        ],
108        "bucketName": "DOC-EXAMPLE-BUCKET1",
109        "versioning": [

```

```

110     ""
111   ],
112   },
113   "responseElements": null,
114   "additionalEventData": {
115     "SignatureVersion": "SigV4",
116     "CipherSuite": "ECDHE-RSA-AES128-SHA",
117     "AuthenticationMethod": "AuthHeader",
118   },
119   "requestID": "07D681279BD94AED",
120   "eventID": "f2b287f3-0df1-4961-a2f4-c4bdfed47657",
121   "eventType": "AwsApiCall",
122   "recipientAccountId": "11112223333"
123 }
124 ]
125 }

```

S3 server access log exemple

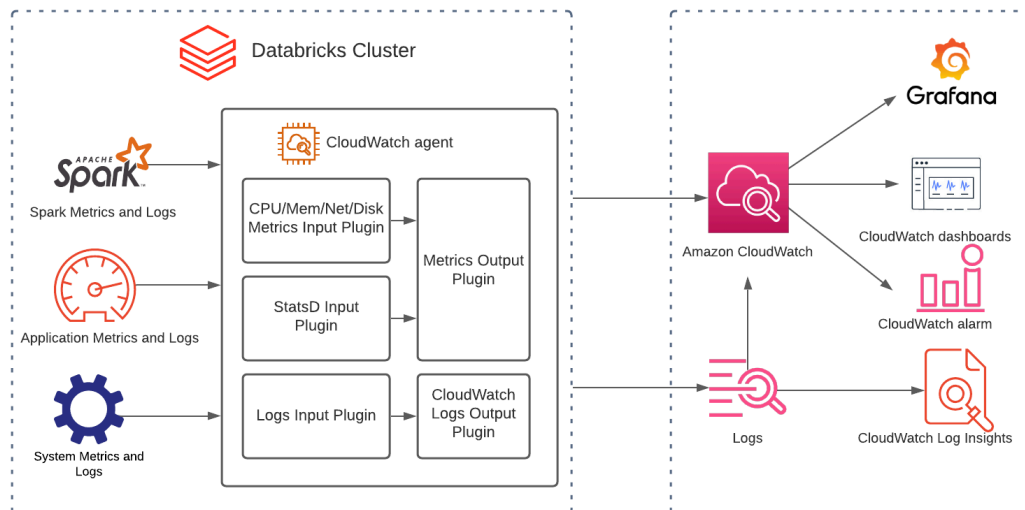
```

1 79a59df900b949e55d96a1e698fbacedfd6e09d98eacf8f8d5218e7cd47ef2be DOC-EXAMPL
2 79a59df900b949e55d96a1e698fbacedfd6e09d98eacf8f8d5218e7cd47ef2be DOC-EXAMPL
3 79a59df900b949e55d96a1e698fbacedfd6e09d98eacf8f8d5218e7cd47ef2be DOC-EXAMPL
4 79a59df900b949e55d96a1e698fbacedfd6e09d98eacf8f8d5218e7cd47ef2be DOC-EXAMPL
5 79a59df900b949e55d96a1e698fbacedfd6e09d98eacf8f8d5218e7cd47ef2be DOC-EXAMPL

```

EC2

Both APM and logs can be collected using the Cloudwatch agent. The Cloudwatch agent should also be able to pickup Spark metrics and forward these as well.



This solution should be able to cover all of our needs when it comes to our computations clusters. While the basic APMs such as CPU, memory and network information are well documented as to what the agent can collect it is not specific what Spark metrics are collected and that would need further testing.

The configuration process does not seem complicated.

- Create a IAM policy for the Cloudwatch agent to be able to reach the Cloudwatch servers
- Add a init script to our Databricks clusters
- Query the data from Cloudwatch

Networking

AWS gives us the flow logs for our VPC here is an example :

```
1 | 2 123456789010 eni-1235b8ca123456789 172.31.16.139 172.31.16.21 20641 22 6  
2 | 2 123456789010 eni-1235b8ca123456789 172.31.9.69 172.31.9.12 49761 3389 6 2
```

Network address usage for each VPC can be forwarded to Cloudwatch.

AWS Firewall flow logs can also be forwarded to Cloudwatch

Conclusion

Fivetran

The tools currently in place allows us to monitor and troubleshoot but only for over the span of 7 days. For us to be able to go over the 7 days of data present from within the dashboard we would need one of these two options.

Send the logs to Cloudwatch so we can keep more than 7 days and potentially implement alerts if needed.

If we are interested in metrics and being able to have all the metadata/metrics about our Fivetran setup then forwarding the logs to the databricks destination makes sense.

These choices are not mutually exclusive and could be used together.

Suggested action items

- Integrate Fivetran with Cloudwatch so that we can monitor it for more than 7 days
- Decide if we want analytical metrics for Fivetran, if so activate them in Fivetran so that they are saved within Databricks

Databricks

Currently there is no way to centralize the monitoring to be able to have a quick dashboard that gives us health checks of our infrastructure.

Investigations have to be done on a per cluster or per job basis which can be problematic since we have multiple workspace to support with more planned in the future.

The other problem is the fact that the most important metrics (spark-ui, ganglia, logs) are only available for a month. If we wanted to see the evolution of our job run times or query performance we would be severely limited. Also even if kept during the 30 days period some aspects like ganglia does not give us the full functionality to be able to drill down onto the APMs of a cluster that is terminated.

Further levels of logging notably the audit logging needs to be configured to make sure that we can track user actions. If we want to enable audit logging for the hive metastore we also need to implement Unity Catalog.

Suggested action items

- Create a S3 bucket for audit logging
- Activate Databricks audit logging
- Activate unity catalog
- Activate hive audit logging
- Integrate the Cloudwatch agent to our Databricks clusters
- Validate that we can access/visualize Spark metrics from a terminated cluster

AWS

AWS has good internal tools to be able to respond to our monitoring needs. Most of the logs and metrics can be forwarded to Cloudwatch and centralize everything.

Suggested action items

- Validate with Devops what they are already monitoring and if New Relic is best suited for some of our needs.
- Activate Cloudtrail in all regions
- Activate Cloudtrail log file validation
- Forward Cloudtrail logs to Cloudwatch
- Configure Cloudwatch log retention policy to at least 365 days
- Encrypt Cloudtrail and Cloudwatch logs
- Enable S3 server access logging to a S3 bucket
- Enable S3 object level logging
- Implement metrics and alarms based on security audit and our monitoring needs.

I believe we should centralize most of the monitoring as possible within Cloudwatch.

We would finally have one centralize place where data from Fivetran, Databricks and AWS would all be consolidated.

Cloudwatch offers what we need so far in terms of functionality such as providing a query language to filter our logs, alerts based on log content or metrics, dashboard.

References :

<https://aws.amazon.com/blogs/mt/how-to-monitor-databricks-with-amazon-cloudwatch/>

 [Metrics collected by the CloudWatch agent - Amazon CloudWatch](#)

 [CloudWatch Logs Insights language query syntax - Amazon CloudWatch L](#)
[ogs](#)

 [Supported logs and discovered fields - Amazon CloudWatch Logs](#)

 [Compute configuration reference](#) | [Databricks on AWS](#)

 [Audit log reference](#) | [Databricks on AWS](#)

 [Billable usage log schema \(legacy\)](#) | [Databricks on AWS](#)

<https://towardsdatascience.com/how-to-access-the-spark-ui-without-setting-up-a-spark-history-server-674570880aac>

 [Deliver and access billable usage logs \(legacy\)](#) | [Databricks on AWS](#)